

Kilo / Workout tracker

A cumulative .NET 10 implementation workbook

Controllers / Dapper / PostgreSQL / Clerk / Docker Compose / Linux

Rebuilt October 6, 2026. Integer IDs. Imperial defaults with explicit lb/kg weights. Routine templates stay separate from workout snapshots.

This is a revised implementation plan, not a completed application. The original PDF supplies the product requirements; its code recipes are reference material, not instructions to execute. This revision supersedes those recipes.

How to use this guide

Build slices 01-20 in order. Each slice states what already exists, the new work, the implementation sequence, and a concrete verification gate. Earlier code remains in place unless a later slice explicitly changes it. No N-number lookup is required.

Slice 01 contains complete foundation recipes. Later snippets illustrate the stated change and are labeled when partial. The API catalog and migration reference describe the final product; they are not a request to implement future slices early.

Use each slice's fillable progress panel for implementation, verification, status, review date, and evidence. Mark Done only when its gate passes. Save the filled PDF in a reader that supports AcroForms.

What changed

- Program.cs composes the host; focused IServiceCollection extensions own API, persistence, and later Clerk registrations.
- Asp.Versioning.Mvc supplies URL-segment versioning. It replaces the hand-built ApiV1Controller routing base.
- Native MVC validation returns 400. The previous custom 422 split is intentionally removed before the first client is built.
- Native health checks replace a custom health controller. Native Npgsql connections and transactions replace DapperDb/DbSession wrappers.
- SQL migrations grow with the slices. There is one fresh-install path, not separate fresh and legacy ALTER scripts mixed together.
- Feature folders grow inside one API project. A separate migrator and a focused test project are justified; speculative architecture layers are deferred.

Development order

Slice	Outcome	Builds on
01	API foundation and migration pipeline	Existing Kilo projects
02	API and migrator release images	01
03	Repeatable local Compose stack	02
04	Clerk identity and preferences	01-03
05	Private exercise library and brands	04
06	Routine templates	05
07	Ordered exercise placements	06
08	Planned sets and explicit weights	07
09	Weekly schedule	08
10	Atomic, idempotent workout snapshots	09
11	Performed sets and corrections	10
12	Finalization and paged history	11
13	Previous performance	12
14	Persistent rest deadline	11-13
15	Reordering and archive rules	14
16	Linux host and restricted DB roles	15
17	HTTPS proxy and browser access	16
18	Repeatable deployment	17
19	Backup and restore	18
20	Recovery, diagnostics, release gate	19

Milestones

After 03: runnable development stack. After 08: complete templates. After 12: usable workout logging API. After 15: complete workout API. After 20: verified single-host deployment.

The frontend is a separate deliverable. The guide retains client behaviors for units and countdowns, but does not claim to build a browser UI. Client QA must use a real client when one is available.

Original weekly plan

Wednesday (3): Upper A. Thursday (4): Lower A. Friday (5): Abs and Arms. Saturday (6): Upper B. Sunday (7): Lower B. Monday and Tuesday may remain unassigned. Starting a workout is never restricted to its scheduled weekday.

Architecture and shared standards

One API, feature folders, explicit dependencies

Keep the existing Kilo and Kilo.Migrations project names. Start with Hosting for service registration and readiness. Add Features/Me, Exercises, Routines, Schedule, and Sessions only when their slice begins. Keep request/response contracts, SQL, and concrete services beside their feature. Move genuinely shared WeightDto or row mapping into a small Shared folder when a second feature needs it.

Controllers bind requests and translate outcomes into HTTP. Concrete repositories own SQL. Introduce a concrete workflow service for a multi-step transaction, not for forwarding one method call. Dapper already performs mapping; do not add AutoMapper, MediatR, generic repositories, a universal Result type, or four Clean Architecture projects.

HTTP and validation

Business routes use `api/v{version:apiVersion}/...`, with `[ApiController]`, `[ApiVersion(1.0)]`, and relative action routes. `GET /health` is anonymous and unversioned. Explicit routes avoid inheritance surprises. From slice 04 onward, a fallback authorization policy protects business routes by default.

Use camelCase JSON and int IDs. A route uses `{id:int:min(1)}`; invalid route shapes do not match (404). MVC binding and DataAnnotations validation return 400 `ValidationProblemDetails`, including invalid semantic input. Use `IValidatableObject` only for cross-field checks; validate ownership and state in the transaction. Keep implicit required validation enabled.

Created: 201 with Location. Read/update: 200. Archive/delete: 204. Missing/invalid token: 401. Authenticated policy denial: 403. Missing/foreign/nested mismatch: 404. Lifecycle, archive, or position conflict: 409. Known dependency outage: 503. Unexpected fault: sanitized 500. Use `ProblemDetails` for errors; status-code middleware fills otherwise empty error responses. Do not treat every database exception as an outage.

Database and time

Pass `CancellationToken` through every async operation. Open/dispose a pooled `Npgsql` connection for each operation. Every Dapper `CommandDefinition` has the operation token, a bounded timeout, and the explicit transaction when one exists. All tenant queries include the verified local `userId`. Child foreign keys enforce ownership as a second defense.

Postgres identity columns generate positive integer keys; gaps are normal. Clerk subjects and retry keys remain text. Use `timestampz` for UTC instants and date for optional scheduled dates. `Npgsql` maps `timestampz` to UTC `DateTime`; map dates explicitly to `DateOnly` at the repository boundary. Use `TimeProvider` for application-generated timestamps, including deterministic timer tests.

Standards basis: Microsoft controller behavior [1], `Npgsql` lifetime guidance [4], and Dapper APIs [5].

Growth map and migration policy

Add only the files the current slice uses

```
Kilo.slnx
global.json
Kilo/
  Program.cs
  Hosting/
    ApiServiceCollectionExtensions.cs
    PersistenceServiceCollectionExtensions.cs
    PostgresReadinessCheck.cs
  Features/
    # added feature by feature
Kilo.Migrations/
  Program.cs
  Migrations/
    001_users.sql
Kilo.Tests/
  # foundation integration checks
```

One ordered, additive schema path

First used	Migration	Adds
01 / 04	001_users.sql	Identity and preference storage
05	002_exercises.sql	Exercise library, optional brands
06	003_routines.sql	Routine metadata
07	004_routine_exercises.sql	Owned ordered placements
08	005_routine_sets.sql	Planned sets and weight pairs
09	006_routine_schedule.sql	One routine per user/weekday
10	007_sessions.sql	Sessions and complete snapshots
12	008_history_indexes.sql	History and previous-session indexes
14	009_rest_timer.sql	Timer columns, checks, child reference

Archive columns and active-position indexes belong to their original table migrations. Archive endpoints and reorder behavior arrive in 15. Session result and lifecycle columns belong to 007 so a session is a complete persisted model in 10; result writes arrive in 11. Timer columns arrive in 14.

Run all outstanding embedded scripts through DbUp, in filename order, transactionally per script, with a migration journal. Repeating the migrator means no applied script reruns; it does not mean adding IF NOT EXISTS everywhere. Never modify or rename a script already applied to a database you retain. Run only one migrator per deployment.

The users baseline is deliberately small and concrete: it proves the schema pipeline and is reused in 04. No exercise, routine, or workout tables are needed for 01. Readiness checks connectivity to the configured database; deployment additionally gates on migration success and an authenticated smoke test.

This is a new app plan. Legacy UUID-to-int imports and pounds-only upgrade scripts are deferred until a real retained legacy database is identified. If one exists, stop before replacing it, inventory its schema, back it up, and design a mapped migration.

Slice 01 - Foundation scope and setup

Outcome

A .NET 10 controller host, package-managed API versioning, pooled persistence, sanitized errors, a public readiness endpoint, and a separate repeatable migration command. No Clerk configuration or feature repository is required yet.

Starting point in this workspace

Kilo.slnx already references Kilo and Kilo.Migrations. Their current source is unfinished: Program.cs contains duplicate registrations/builds and references future types; global.json uses runtime-style 10.0.0 instead of a valid SDK version. Preserve any useful project setup and replace startup coherently when implementing this slice.

Build sequence

1. Correct global.json and confirm dotnet --version selects a stable 10.0 SDK. Keep net10.0, Nullable, and ImplicitUsings in both projects.
2. Keep Microsoft.AspNetCore.OpenApi in Kilo. Add Asp.Versioning.Mvc; keep Dapper and Npgsql. JwtBearer is used in 04. Do not add the deprecated Microsoft.AspNetCore.Mvc.Versioning package.
3. Keep dbup-postgresql and Microsoft.Extensions.Hosting in the migrator. The web SDK already supplies configuration APIs; remove redundant web-project Microsoft.Extensions.Configuration package references when safe.
4. Create the two registration extensions and readiness check shown on the next pages. Program.cs only composes them.
5. Add 001_users.sql and embed Migrations/*.sql. Configure ConnectionStrings:Postgres separately for each executable.
6. Build, run migrations twice against a disposable Postgres database, then exercise readiness. Add the focused checks described in the gate.

SDK selection

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature",
    "allowPrerelease": false
  }
}
```

10.0.100 is a valid minimum installed SDK, not a claim about the newest SDK. For reproducible release builds, select an exact approved SDK and compatible container digest; with locked package graphs, use rollForward=disable in CI. Record resolved package versions and commit packages.lock.json. Never copy a guessed future patch number. SDK selection: [6].

Slice 01 - Program.cs and API services

Kilo/Program.cs - complete slice-one composition

```
using Kilo.Hosting;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddKiloApi();
builder.Services.AddKiloPersistence(builder.Configuration);

var app = builder.Build();
app.UseExceptionHandler();
app.UseStatusCodePages();

if (app.Environment.IsDevelopment())
    app.MapOpenApi();

app.MapHealthChecks("/health").AllowAnonymous();
app.MapControllers();
app.Run();

public partial class Program { }
```

Hosting/ApiServiceCollectionExtensions.cs - complete registration

```
using Asp.Versioning;

namespace Kilo.Hosting;

public static class ApiServiceCollectionExtensions
{
    public static IServiceCollection AddKiloApi(
        this IServiceCollection services)
    {
        services.AddControllers();
        services.AddProblemDetails();
        services.AddOpenApi();
        services.AddSingleton<TimeProvider>(TimeProvider.System);
        services.AddApiVersioning(options =>
        {
            options.DefaultApiVersion = new ApiVersion(1, 0);
            options.AssumeDefaultVersionWhenUnspecified = false;
            options.ReportApiVersions = true;
            options.ApiVersionReader = new UrlSegmentApiVersionReader();
        }).AddMvc();
        return services;
    }
}
```

The package owns version matching; URL segments still require an explicit version. `DefaultApiVersion` does not make `/api/exercises` valid. `Asp.Versioning.Mvc` is the current .NET Foundation project descended from the Microsoft-named library [2, 3]. Add its `ApiExplorer` companion only when separate per-version documents are needed. Development OpenAPI is the built-in Microsoft implementation; no UI package is required for 01.

Keep middleware ordering visible. Do not add a second `app.Build()`, a self-check branch to production startup, or one extension per registration line.

Slice 01 - Persistence registration

Hosting/PersistenceServiceCollectionExtensions.cs

Complete registration. DI owns one data source; requests own short-lived connections. No singleton NpgsqlConnection and no custom connection/transaction wrapper.

```
using Npgsql;

namespace Kilo.Hosting;

public static class PersistenceServiceCollectionExtensions
{
    public static IServiceCollection AddKiloPersistence(
        this IServiceCollection services, IConfiguration config)
    {
        var connectionString = config.GetConnectionString("Postgres");
        if (string.IsNullOrEmpty(connectionString))
            throw new InvalidOperationException(
                "ConnectionStrings:Postgres is required.");

        NpgsqlConnectionStringBuilder settings;
        try
        {
            settings = new NpgsqlConnectionStringBuilder(connectionString);
        }
        catch (ArgumentException)
        {
            throw new InvalidOperationException(
                "ConnectionStrings:Postgres is invalid.");
        }

        if (string.IsNullOrEmpty(settings.Host)
            || string.IsNullOrEmpty(settings.Database))
            throw new InvalidOperationException(
                "Postgres Host and Database are required.");

        services.AddSingleton<NpgsqlDataSource>(provider =>
        {
            var dataSource = new NpgsqlDataSourceBuilder(connectionString);
            dataSource.UseLoggerFactory(
                provider.GetRequiredService<ILoggerFactory>());
            return dataSource.Build();
        });
        services.AddHealthChecks().AddCheck<PostgresReadinessCheck>(
            "postgres", timeout: TimeSpan.FromSeconds(3));
        return services;
    }
}
```

Configuration errors name the key, not its value. Supply explicit connection/command timeouts, for example Timeout=3;Command Timeout=15 in development configuration. Do not enable SQL parameter-value logging. The data source pools connections and is disposed by the container [4].

A well-formed connection string is configuration validity; an unreachable server is readiness failure. Do not perform schema writes in this registration factory.

Slice 01 - Native readiness and versioned controllers

Hosting/PostgresReadinessCheck.cs

Complete class. The linked token bounds pool acquisition and the query. Do not leak database error details into the response or return a caught exception in HealthCheckResult.

```
using Microsoft.Extensions.Diagnostics.HealthChecks;
using Npgsql;

namespace Kilo.Hosting;

public sealed class PostgresReadinessCheck(NpgsqlDataSource source)
    : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken ct = default)
    {
        using var deadline = CancellationTokenSource
            .CreateLinkedTokenSource(ct);
        deadline.CancelAfter(TimeSpan.FromSeconds(3));
        try
        {
            await using var connection =
                await source.OpenConnectionAsync(deadline.Token);
            await using var command = connection.CreateCommand();
            command.CommandText = "SELECT 1";
            command.CommandTimeout = 3;
            await command.ExecuteScalarAsync(deadline.Token);
            return HealthCheckResult.Healthy();
        }
        catch (NpgsqlException)
        {
            return HealthCheckResult.Unhealthy("Database unavailable.");
        }
        catch (OperationCanceledException)
        {
            return HealthCheckResult.Unhealthy("Database check timed out.");
        }
    }
}
```

GET /health uses the framework's short Healthy/Unhealthy text response and 200/503 status. It is the documented operational exception to JSON ProblemDetails. It needs no custom HealthController or health-check dependency package [7].

Controller pattern - introduced with the first business controller in 04

```
[ApiController]
[ApiVersion(1.0)]
[Route("api/v{version:apiVersion}/me")]
public sealed class MeController : ControllerBase
{
    // Slice 04 supplies the actual dependencies and actions.
}
```

This pattern is partial, not a slice-one endpoint. There are no exercise routes to test yet. Test package matching with a test-only controller in 01; test real protected routes after 04. Positive ID constraints arrive with resource actions. No routing base class is needed.

Slice 01 - Separate migration executable

Kilo.Migrations/Program.cs - complete entry point

```
using DbUp;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Hosting;

var builder = Host.CreateApplicationBuilder(args);
var connectionString = builder.Configuration
    .GetConnectionString("Postgres");
if (string.IsNullOrEmpty(connectionString))
{
    Console.Error.WriteLine("ConnectionStrings:Postgres is required.");
    return 1;
}

try
{
    var upgrader = DeployChanges.To
        .PostgresqlDatabase(connectionString)
        .WithScriptsEmbeddedInAssembly(typeof(Program).Assembly)
        .WithTransactionPerScript()
        .Build();
    var result = upgrader.PerformUpgrade();
    if (!result.Successful)
    {
        Console.Error.WriteLine("Migration failed; release not activated.");
        return 1;
    }
}
catch (Exception)
{
    Console.Error.WriteLine("Migrator failed; check protected diagnostics.");
    return 1;
}

Console.WriteLine("Migrations complete.");
return 0;
```

The top-level catch is an executable failure boundary. Avoid logging raw connection errors, credentials, or data-bearing SQL. Add safe script identity/timing through DbUp's logging hook when operational diagnostics are implemented. DbUp defaults to no transaction; WithTransactionPerScript is intentional [8]. A failed later script may leave earlier successful scripts journaled. Correct the unapplied failure and rerun; do not claim the whole release is one transaction.

Kilo.Migrations.csproj additions and baseline

```
<ItemGroup>
  <EmbeddedResource Include="Migrations/*.sql" />
</ItemGroup>
```

001_users.sql creates users with integer identity PK, unique nonblank clerk_user_id, time_zone default UTC, measurement_system default imperial with an imperial/metric check, and created_at timestampz default now(). The exact SQL appears in the migration reference. No manual BEGIN/COMMIT inside scripts; DbUp owns the transaction.

The API uses an application role; the migrator uses a DDL role in production. The API never calls PerformUpgrade. The empty database itself is provisioned by local Postgres/Compose, not by every serving process.

Slice 01 - Configuration and verification gate

Configure each executable independently

Both projects already have distinct UserSecretsId values. Configure the same key for each; a console migrator does not inherit the API's secret store. These commands use placeholders, not committed credentials.

```
$env:DOTNET_ENVIRONMENT = "Development"
dotnet user-secrets set "ConnectionStrings:Postgres" "<connection>" --project Kilo
dotnet user-secrets set "ConnectionStrings:Postgres" "<connection>" --project Kilo.Migrations
dotnet restore Kilo.slnx --use-lock-file
dotnet build Kilo.slnx --no-restore
dotnet run --project Kilo.Migrations
dotnet run --project Kilo.Migrations
dotnet run --project Kilo --no-launch-profile --urls http://127.0.0.1:8080
Invoke-WebRequest http://127.0.0.1:8080/health
```

For the native Windows development process, Host=localhost targets an explicitly available local DB. In containers, use Host=db. Environment override: ConnectionStrings__Postgres. User secrets are development storage, not an encrypted production vault. Supply production credentials through restricted host configuration.

The console migrator has no web launch profile: explicitly selecting Development is what enables its development user-secret loading. Production images use Production configuration with externally supplied settings.

Package pins for the checked foundation

```
dotnet add Kilo package Asp.Versioning.Mvc --version 10.2.1
dotnet add Kilo package Microsoft.AspNetCore.OpenApi --version 10.0.12
dotnet add Kilo package Npgsql --version 10.0.3
dotnet add Kilo package Dapper --version 2.1.89
dotnet add Kilo.Migrations package dbup-postgresql --version 7.0.1
dotnet add Kilo.Migrations package Microsoft.Extensions.Hosting --version 10.0.12
```

These are verified package pins for recipe checks, not automatic update promises. Pin a patched OpenAPI package rather than keeping the template's 10.0.0 transitive dependency warning. Review package audit output during restore and update the lock file deliberately. Current package identities/targets: [14].

Slice 01 - Foundation verification

One focused runnable integration suite

Create Kilo.Tests with the chosen stable xUnit runner and Microsoft.AspNetCore.Mvc.Testing 10.x. Use WebApplicationFactory<Program> and a disposable real Postgres database supplied by test configuration. Reuse that small harness for future slices. No mocks of Npgsql or an in-memory substitute for relational behavior.

- Migrate an empty database; assert users and the DbUp journal exist. Insert a fixture and rerun; assert the fixture and journal row count are unchanged.
- Start with valid settings; GET /health returns 200. Stop the test DB or point to an unreachable endpoint; it returns 503 within the bounded check deadline plus transport overhead. Bring DB back; readiness recovers.
- Missing/invalid connection configuration fails with a key-only diagnostic. Capture logs and assert test credentials are absent.
- Add a test-assembly v1 probe controller through MVC ApplicationPart. /api/v1/probe resolves; unversioned and unsupported-version paths do not reach the v1 action. Assert the package's documented 404 for unmatched URL versions. Keep this probe out of production.
- Force the second command of a test transaction to fail; disposal rolls back the first. Exercise cancellation while a query is active and show a later pooled operation still succeeds.
- Run the migrator with invalid configuration/failing unapplied SQL; assert nonzero exit. Confirm the serving executable never runs migrations.

Done means

Build passes; migration repeatability, sanitized failure, rollback/cancellation, readiness, and package routing checks pass. Health is the only application endpoint. Do not mark future schema, Clerk, Docker, or feature CRUD complete.

PROGRESS / SLICE 01

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 02 - Build release images

Already exists

01 supplies two executables, embedded SQL, package locks, and valid SDK selection. Keep separate API and migrator images; use the existing two Dockerfiles as a starting point.

Add and implement

1. Use a .NET 10 SDK build stage to restore in locked mode and publish Release with --no-restore. Copy solution, SDK selection, project files, and lock files before sources for caching.
2. API final stage uses aspnet:10.0; migrator uses runtime:10.0. Run both as the image's unprivileged APP_UID. The API listens on 8080 through ASPNETCORE_HTTP_PORTS.
3. SQL is embedded in the migrator DLL; no runtime script-folder copy or API migration command is required. Both images carry one release identifier.
4. Keep secrets, .git, .idea, bin, obj, tmp, and output artifacts out of the Docker build context. Use .dockerignore. Build requires package restore access but no running database.
5. Pin approved release base-image digests and build for the host CPU architecture. Preserve runtime CA certificates and globalization/timezone data needed for Clerk and IANA IDs.

Minimal publish shape - illustrative Dockerfile body

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore Kilo/Kilo.csproj --locked-mode
RUN dotnet publish Kilo/Kilo.csproj -c Release --no-restore \
    -o /out /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
ENV ASPNETCORE_HTTP_PORTS=8080
USER $APP_UID
COPY --from=build /out .
ENTRYPOINT ["dotnet", "Kilo.dll"]
```

Use the analogous project path, runtime base, and Kilo.Migrations.dll for the migrator. The above favors clarity; the existing project-copy restore cache can stay. No orchestrator or registry abstraction is required.

Slice 02 - Verify images

Gate

- Build both images from a clean source checkout without bin/obj or a database. Verify embedded migration discovery in the migrator image.
- Run on the intended Linux architecture; inspect UID, listening port, and environment-only settings.
- Scan layers/build context for secret files. Verify outbound trusted HTTPS works from the runtime used by the API.
- Send SIGTERM during a real transactional write after that feature exists; require complete commit or rollback. For now verify host shutdown and container exit.

Carry forward

Record image tags/digests and the build command. Later releases use the same image pair, not a new packaging strategy. The migration executable exits once; the API runs until stopped.

Acceptance

02.1 Clean image builds succeed. 02.2 Images run on target architecture. 02.3 Runtime is non-root. 02.4 Trusted HTTPS/globalization dependencies remain available. 02.5 SIGTERM stops the host cleanly; data-integrity shutdown is rechecked in 20.

No Dockerfile-only unit tests. Verify the actual images.

PROGRESS / SLICE 02

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 03 - Local Docker Compose

Already exists

02 supplies API/migrator images; 01 supplies the readiness route and one shared connection-string key. No application code change is needed.

Add and implement

1. Keep services db, migrations, api. Postgres health uses pg_isready. Migrations waits for service_healthy; API waits for service_completed_successfully. Migrations has no restart loop; API/db use appropriate long-running restart policies.
2. Use POSTGRES_DB=kilo, POSTGRES_USER, and a locally supplied password. The current POSTGRES_DATABASE setting is ineffective. For Postgres 18, mount the persistent volume at /var/lib/postgresql, matching its image layout [9].
3. Use Host=db;Database=kilo in container connection strings. Inject credentials from ignored development configuration. Configure each executable separately.
4. Put development port publishing in an override: API 127.0.0.1:8080:8080; publish DB to loopback only if Rider/native API tests need it. Production has neither direct API nor DB port publishing.
5. Start the full stack; migrations are a fresh one-shot run when explicitly requested. Do not assume an old completed migration container ran scripts from a new image; deployment handles this in 18.

Compose dependency shape - partial snippet

```
migrations:
  depends_on:
    db:
      condition: service_healthy
api:
  depends_on:
    migrations:
      condition: service_completed_successfully
```

This is the dependency fragment, not a full compose.yaml. Include the established images/builds, environment, volume, and DB health check when applying it.

Persistent-state rule

Container recreation is routine. Removing the named database volume is a data reset and must be intentional. Changing POSTGRES_DB or POSTGRES_PASSWORD after initialization does not rewrite an existing database or its roles [9].

Slice 03 - Verify the local stack

Gate

- Start with a disposable empty volume; observe db healthy, migrations exits 0, then API becomes ready.
- Create a users fixture through test SQL, recreate containers without deleting volumes, and assert it survives.
- Make a new unapplied test migration fail. API startup is blocked; failure is visible. Repair only the unapplied script and rerun it freshly.
- Confirm container Host=db networking and loopback-only development ports.
- Inspect the base production Compose configuration: no published DB/API ports and no hardcoded development password.

Acceptance

03.1 Fresh startup succeeds in dependency order. 03.2 Container service networking works. 03.3 Data survives recreation. 03.4 Migration failure blocks API start. 03.5 Development access is local.

No Compose-only unit suite. Store one reproducible startup/failure check with the run instructions; reuse it for releases.

Next change

04 extends API registration with identity and authorization. It does not rewrite persistence, the health route, or the migration host.

PROGRESS / SLICE 03

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 04 - Clerk identity and preferences

Already exists

01's users table already holds Clerk subject, timezone, and measurement preference. Reuse pooled connections, MVC validation, and v1 versioning. Health stays anonymous.

Add and implement

1. Add Hosting/ClerkServiceCollectionExtensions.cs with AddKiloClerk(configuration). Bind ClerkOptions (Issuer, optional Audience, AuthorizedParties), validate HTTPS issuer and nonempty allowed origins with ValidateOnStart, and register JwtBearer. Do not fetch or parse JWTs yourself.
2. Let the handler retrieve/cache trusted signing metadata and rotate keys. Set MapInboundClaims=false; verify issuer, lifetime, signing key, and intended algorithm. Validate audience when your Clerk token setup defines it. Require nonblank verified sub and allowed azp in OnTokenValidated [10]. No Clerk secret API key is needed just for signature verification.
3. Register a fallback policy requiring authenticated users, and a named frontend CORS policy with explicit configured origins. CORS permission and token authorized-party validation are separate checks even if their configured origins match.
4. Add Features/Me with CurrentUser, preferences requests/responses, SQL, and MeController. Resolve sub from the authenticated principal only. Use INSERT ... ON CONFLICT ... RETURNING id to provision once; cache the resolved ID inside the scoped resolver for that request.
5. GET /me returns the local profile. PUT /me/preferences replaces both fields. Validate imperial/metric and an actual resolvable IANA timezone (UTC accepted); verify Linux runtime support. Never change stored workout weights on preference updates.

Program.cs delta after the two existing registrations

```
builder.Services.AddKiloClerk(builder.Configuration);  
// After Build and exception/status-code middleware:  
app.UseRouting();  
app.UseCors("frontend");  
app.UseAuthentication();  
app.UseAuthorization();
```

The delta has two insertion points, not a block to paste at the bottom. Keep registration before Build and middleware before endpoint mapping. OpenAPI in development and /health must opt out with AllowAnonymous when the fallback policy is enabled.

Routes: GET /api/v1/me; PUT /api/v1/me/preferences. New users default to UTC and imperial; client weight-input default is lb. No new SQL migration.

Slice 04 - Verification and auth outage policy

Gate

- Valid test JWT provisions one local integer ID; concurrent first requests produce exactly one users row.
- Missing, expired, tampered, wrong-issuer, missing-sub, disallowed-algorithm, wrong audience (when configured), and wrong azp return 401 and create no account.
- Valid identity plus unmet authorization policy returns 403. Foreign-data tests begin in 05.
- Rotate fixture signing keys and verify metadata refresh. An outage with usable cached metadata still verifies valid cached-key tokens.
- No usable trusted metadata: deny the request. Return 503 only when metadata retrieval failure is positively identified as the cause. Unknown key or invalid signature alone remains 401. Preserve this distinction in one auth-specific failure hook; do not broadly convert authentication errors to 503.
- PUT America/Phoenix + metric, then imperial; verify persistence. Missing/unknown preference or unknown timezone returns 400 and changes nothing.
- Anonymous /health and allowed CORS preflight still work. A disallowed browser origin receives no CORS permission.

Test shape

Extend the existing WebApplicationFactory/Postgres fixture. Ordinary feature tests may use a test authentication handler; keep a separate small real JWT/JWKS fixture for signature, claim, key rotation, and outage behavior. Do not retest the internals of Microsoft's JWT library with a large mock suite.

Acceptance

04.1 Verified identity resolves a local account. 04.2 Concurrent provisioning is race-safe. 04.3 Invalid tokens fail. 04.4 Claim restrictions hold. 04.5 Rotation and diagnosed outages behave safely. 04.6 Imperial is the default. 04.7 Preference changes persist without rewriting values.

Carry forward

Every feature receives local userId from CurrentUser, never from a request body. Business controllers use the established v1 attributes. For created resources include the route version when generating Location with CreatedAtAction.

PROGRESS / SLICE 04

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 05 - Exercise library

Already exists

Authenticated local users, scoped CurrentUser, v1 controllers, pooled connections, and a real DB test harness. Add 002_exercises.sql from the migration reference.

Add and implement

1. Create Features/Exercises with ExerciseWriteRequest, ExerciseDto, ExerciseRepository, and ExercisesController. Register the concrete repository as scoped in a small feature registration extension once several feature registrations need grouping.
2. Keep DTO validation with the request. Require a nonblank trimmed name; normalize optional description to empty string. Validate the trimmed brand in IValidatableObject (rather than applying a raw-string length attribute), then normalize omitted/null/blank brandName to null and limit a nonempty brand to 100 characters. PUT omission clears it.
3. Use parameterized INSERT ... RETURNING and explicit projections/aliases. Every list/detail/update includes user_id=@userId. Do not map nested response objects directly through Dapper.
4. Implement list, detail, create, metadata replacement. Exclude archived rows by default; includeArchived is an explicit query option. Updates to an owned archived exercise return 409; absent/foreign IDs return 404. Archive itself arrives in 15.
5. Use CreatedAtAction for 201 Location, passing version=1 and id. Verify it round-trips through real routing.

Repository operation shape - partial method body

```
await using var connection = await source.OpenConnectionAsync(ct);
var command = new CommandDefinition(
    """
        SELECT id AS "Id", name AS "Name", description AS "Description",
               brand_name AS "BrandName", archived_at AS "ArchivedAt"
        FROM exercises WHERE id = @id AND user_id = @userId
    """,
    new { id, userId }, commandTimeout: 15, cancellation_token: ct);
return await connection.QuerySingleOrDefaultAsync<ExerciseDto>(command);
```

This uses Dapper and Npgsql namespaces and the injected NpgsqlDataSource. Repeat a short native connection scope for simple operations; do not rebuild DapperDb. A later multi-command workflow owns one connection and transaction for its whole operation.

Routes: GET/POST /exercises; GET/PUT /exercises/{id}. Brand is optional free-form equipment text, not a separate catalog or foreign key.

Slice 05 - Verify exercise CRUD

Gate

- Create Bench Press and Chest Press with Unicode/apostrophes/multiline descriptions. Verify 201, positive numeric ID, camelCase DTO, and a working versioned Location.
- Create null, blank, padded, 100-character and 101-character brands. Verify normalization, accepted boundary, and 400 failure boundary.
- List empty accounts and separate accounts. Foreign detail/update is 404 and changes nothing.
- PUT without brandName clears a previous brand. Update preserves the resource ID and unrelated fields.
- Insert an archived test fixture; default list hides it, includeArchived shows it, update returns 409.
- Malformed JSON, missing required name, and semantic invalid fields return 400 ValidationProblemDetails. /api/exercises and /api/v2/exercises do not invoke v1.

Acceptance

05.1 Create/list works. 05.2 Ownership comes from verified identity. 05.3 Names/descriptions round-trip. 05.4 Lists are private and may be empty. 05.5 Optional brands round-trip. 05.6 PUT can clear brands. 05.7 Brand limits hold.

Carry forward

Keep the same SQL ownership predicate and input validation style in routines. Library metadata remains editable; sessions will later copy historical values instead of joining current descriptions/brands.

PROGRESS / SLICE 05

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 06 - Routine templates

Already exists

Exercise feature patterns and user isolation. Add 003_routines.sql; keep the established startup and authentication code.

Add and implement

1. Add Features/Routines with requests/responses, controller, and concrete SQL repository. Use named operations, not a generic repository shared with exercises.
2. Implement create, private list, detail, and metadata replacement. Require a nonblank trimmed name; description is optional and normalizes to empty string.
3. Detail returns exercises=[] until 07 adds placements. Avoid querying a table that does not exist yet. When children arrive, extend the same detail assembler rather than duplicating the endpoint.
4. Filter archived routines from default lists, allow includeArchived explicitly, and reject metadata edits of owned archived routines with 409. Foreign/missing routine is 404.
5. Add its scoped registration alongside Exercises. One AddKiloFeatures extension can collect concrete feature registrations; grow it with each slice. No service-location or reflection scan is needed.

Routes: GET/POST /routines; GET/PUT /routines/{id}. Creating an empty routine is valid. It cannot start a workout until it contains active planned sets.

Gate

- Create Upper A, Lower A, Abs and Arms, Upper B, Lower B with distinct generated IDs.
- Create empty routines with/without descriptions; verify detail collections are empty rather than null.
- Reject blank names with 400 and no row. Exercise standard missing/foreign detail/update 404 and archived update 409.
- A second account lists none of the first account's routines. Created Location works with the API version.

Acceptance

06.1 Routine create/list works. 06.2 Empty templates are valid. 06.3 Names are validated. 06.4 Routine ownership is enforced.

PROGRESS / SLICE 06

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 07 - Ordered exercise placements

Already exists

Owned exercises and routines. Add 004_routine_exercises.sql with composite owner foreign keys and an active-position unique index. Extend routine details to read ordered placements.

Add and implement

1. Add POST/PUT nested placement actions to the routine feature. Use route name placementId instead of ambiguous exerciseId; the wire path shape stays compatible. Body exerciseId is the library exercise ID.
2. Create distinct placement IDs, allowing the same library exercise more than once. Store position>0, placement-specific description, and defaultRestSeconds>=0. Existing placement exerciseId is immutable; replacement will archive and create a new placement.
3. For each placement write, open a connection and ReadCommitted transaction. Lock the owned routine first, then the owned library exercise when attaching. Require both active. Nested update must prove placement.routine_id equals the route routineId and user_id equals the caller.
4. Return current library exercise name/brand through the routine detail join; store no placement brand column. Sets remain [] until 08.
5. Pass the same transaction and ct to every command. Translate only the named active-position unique constraint violation to 409; let unrelated database faults remain errors.

Transaction shape - partial workflow

```
await using var connection = await source.OpenConnectionAsync(ct);
await using var transaction = await connection.BeginTransactionAsync(ct);
// Lock owned routine; then owned exercise; validate; insert placement.
// Every command uses new CommandDefinition(sql, args, transaction,
//     commandTimeout: 15, cancellationToken: ct).
await transaction.CommitAsync(ct);
```

No inner repository call may open another connection for this workflow. Put transaction SQL in the owning concrete workflow/repository. When several methods actually share a lock operation, extract a small method accepting connection and transaction; do not introduce a generic unit-of-work framework.

Lock ordering starts here: routine parent before library exercise; all code that needs both follows this order. Exercise-only archive will lock just the exercise. Future session mutations use their own parent session lock.

Slice 07 - Verify placements

Gate

- Add positions 1-3 and verify ascending detail order. Give repeated Bench Press placements distinct instructions and IDs.
- Reject nonpositive/fractional positions, negative/noninteger rest, and duplicate active positions. A unique collision is 409; request validation is 400.
- Attach foreign exercise/routine or update a placement under the wrong routine; return 404 and preserve all rows.
- Attach an owned archived exercise or edit an archived routine; return 409. The lock/state check must not only rely on a stale read.
- Change a library brand; both repeated routine placements show the current brand, without writing the placement rows.
- Race two inserts at one position; one commits and one returns 409, with no partial work.

Acceptance

07.1 Configured order is stable. 07.2 Instructions belong to placement. 07.3 Repeated movements retain distinct identity. 07.4 Position/rest bounds hold. 07.5 Parent/owner checks hold. 07.6 Routine details show current brands.

Carry forward

Template child writes lock the routine parent. This makes reorder and archive atomic later. Snapshot creation will use RepeatableRead so multi-query copies see one committed template state.

PROGRESS / SLICE 07

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 08 - Planned sets and units

Already exists

Owned ordered placements and the routine transaction convention. Add 005_routine_sets.sql; extend routine details with ordered sets. Introduce shared WeightDto when both templates and future results use it.

Add and implement

1. Add POST/PUT sets under a route routineId/placementId. Prove the complete routine->placement->set chain, not only ownership of the final ID. Lock the routine before writing.
2. Validate setType warmup/working, position>0, targetRepsMin>0, targetRepsMax>=min. Integer binding rejects fractional reps. Use DataAnnotations plus IValidatableObject for the range relationship.
3. A weight is null or a complete {value,unit} object. Use required nullable decimal?/string input members with [Required] so an omitted value cannot silently become zero; response WeightDto contains decimal/string. Validate value>=0 and unit lb/kg. Reject JSON outside decimal range. No unit inference from a profile.
4. Store value/unit as a nullable pair using separate SQL parameters; the DB check enforces both absent or both present. Store exact original decimal values. Use unconstrained numeric (no fixed-scale rounding), bounded at the API by decimal.
5. Nullable restSeconds inherits the placement default; explicit zero means zero. Resolve this only when a session or extra set is created. Keep the template override nullable.

Wire examples

```
{
  "position": 1,
  "setType": "working",
  "targetRepsMin": 6,
  "targetRepsMax": 8,
  "targetWeight": { "value": 135.5, "unit": "lb" },
  "restSeconds": null
}
```

Read via internal row fields such as TargetWeightValue and TargetWeightUnit, then assemble the nullable WeightDto. Dapper rows are flat; API responses are nested. SQL aliases keep mapping explicit.

Slice 08 - Verify planned sets

Gate

- Two warmup plus three working sets produce five ordered rows.
- Fixed 10-10 and ranged 6-8 targets round-trip. Inverted/nonpositive/fractional ranges return 400.
- Null, zero, 2.5, 45, 135.5 lb and 60 kg round-trip exactly through JSON and SQL.
- Weight object {} or missing value/unit, unknown unit, negative value, invalid set type, and negative rest return 400 without writes. Direct SQL with a half-present value/unit pair is rejected too.
- A null rest override stays null; zero stays zero; 120 stays 120. 10 verifies resolved copies.
- Wrong routine/placement/set chain and foreign rows return 404. Position collisions return 409, atomically.

Acceptance

08.1 One row per ordered set. 08.2 Fixed/ranged targets. 08.3 Null and decimal weights. 08.4 Invalid values rejected. 08.5 Rest inheritance preserved. 08.6 Complete parent relationships enforced. 08.7 Every present weight has an explicit unit.

Carry forward

Client input defaults follow measurementSystem; requests always send the chosen unit. Preference changes affect presentation, never stored values. Do not add conversion logic to save handlers.

PROGRESS / SLICE 08

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 09 - Weekday schedule

Already exists

Owned templates and profile timezone. Add 006_routine_schedule.sql with primary key (user_id,weekday) and owned routine foreign key.

Add and implement

1. Add Features/Schedule with GET /schedule and PUT/DELETE /schedule/{weekday}. Weekday is an integer 1-7, Monday=1 and Sunday=7.
2. PUT verifies/locks the owned active routine, then INSERT ... ON CONFLICT (user_id,weekday) DO UPDATE inside the same transaction. One routine may occupy several days.
3. DELETE clears only the authenticated user's selected weekday and is idempotent (204 even when already empty). Return assigned entries in weekday order, not seven fabricated rows.
4. Calculate an intended user's local day from TimeProvider UTC plus their timezone. Do not use the server's local timezone. In .NET DayOfWeek, Sunday is zero; map it to ISO 7.
5. Do not validate scheduledDate against a weekly assignment at session start. The schedule is a convenience, not a permission to train.

Gate

- Assign the original routines Wednesday-Sunday (3-7). Replace Wednesday twice; one row remains.
- Assign Upper A to Monday and Wednesday; both exist. Clear Wednesday twice; Monday and other users stay unchanged.
- Invalid weekday/binding returns 400 (or 404 if rejected by route constraint); settle the exact action contract in its HTTP test. Use {weekday:int} plus [Range(1,7)] for the documented 400.
- Foreign routine returns 404; owned archived routine returns 409. No schedule row is written.
- Test UTC day boundaries for America/Phoenix and a DST-observing zone.

Acceptance

09.1 ISO mapping is correct. 09.2 One row per user/day. 09.3 Weekday bounds hold. 09.4 Repeated routine assignments work. 09.5 Private reads and owned writes hold.

PROGRESS / SLICE 09

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 10 - Start a workout snapshot

Already exists

Complete templates and transactional SQL. Add 007_sessions.sql. Create Features/Sessions with SessionDto assembly and a concrete StartSession workflow. Do not add repositories that open connections inside that workflow.

Add and implement

1. POST /routines/{id}/sessions receives optional scheduledDate and an Idempotency-Key: 1-128 visible ASCII characters. Validate before opening a transaction. The key is text, not a generated resource ID.
2. In a RepeatableRead transaction, look up (user_id,key). If present, compare original routineId and scheduledDate: same payload returns existing snapshot with 200; different payload returns 409. This replay remains valid even if the routine was subsequently archived.
3. If new, require the owned active routine and at least one active planned set. Copy routine metadata, active ordered placements, current exercise names/brands, ordered targets, both weight columns, and resolved rest. Actual reps/weights/completion timestamps start null.
4. Keep sourceRoutineExercisId for placement matching; never substitute library exercisId. Return 201 with session Location for a new snapshot. GET /sessions/{id} reads the snapshot only.
5. Let errors escape to roll back all rows. Unique-key races require a fresh transaction/snapshot to resolve the persisted key. Bounded retry (for example, at most 3 attempts) applies only to the named start-key unique violation, serialization failure, or deadlock for this atomic workflow. Never retry arbitrary writes automatically.

Why RepeatableRead here

Several SELECT/INSERT statements must see one committed template state, including concurrent brand edits. The routine parent lock convention alone does not serialize independent library metadata updates.

RepeatableRead supplies a consistent snapshot; transaction conflicts restart the whole copy, not just the last statement [11].

Copy expression for rest: COALESCE(routine_sets.rest_seconds, routine_exercises.default_rest_seconds). Copy weight value and unit without conversion. Workouts may start on any day; scheduledDate is optional contextual data.

Slice 10 - Verify snapshots and retries

Gate

- Compare a new session with its routine: copied names, brand, instructions, ordering, set types, reps, weight pairs, resolved rest. Verify all actual fields are null.
- Copy 135.5 lb and 60 kg; switch preference; copied original numbers/units remain identical.
- Same key/payload sequentially and concurrently returns one session. First new response is 201; replay is 200. Changed routine/date with that key is 409.
- The same key from another user is independent. Invalid key is 400. Foreign routine/session is 404.
- Owned archived routine or no active planned sets returns 409 and no snapshot. Replaying a previously successful start still returns it.
- Fail halfway through copy; no parent/child rows survive. Race template edits and brand changes against start; every snapshot matches one committed state.
- Force unique/serialization race; verify a fresh bounded attempt and no duplicate data. Exhausted confirmed transient retry is 503 with safe retry guidance.

Acceptance

10.1 Complete snapshot. 10.2 Exact targets. 10.3 Unperformed actuals. 10.4 Atomic consistent creation. 10.5 Idempotent retries. 10.6 Off-schedule start. 10.7 Unusable template rejection. 10.8 Brand snapshot. 10.9 Original units retained.

Carry forward

Every later session mutation begins by SELECT ... FOR UPDATE on the owned workout_sessions row. This serializes set logging, finalization, extra sets, and timers. Read-only DTO assembly does not acquire a write lock.

PROGRESS / SLICE 10

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 11 - Record and correct sets

Already exists

007 already has result columns and session-state constraints. Reuse SessionDto assembly, explicit weight input, and the parent session transaction convention. No new migration.

Add and implement

1. PUT /sessions/{sessionId}/sets/{setId}/result locks the owned session. Missing/foreign/mismatched chain is 404; non-in_progress session is 409. Validate actualReps>=0, setType, and optional complete weight pair before mutation.
2. Null actualReps means untouched. Recording zero reps still performs the set. On the first result, set completed_at using TimeProvider; corrections preserve that original instant and copied targets.
3. POST /sessions/{sessionId}/exercises/{sessionExerciseId}/sets appends an extra set. Under the same parent lock allocate max(position)+1; targets/results are null; default rest resolves from the copied session exercise unless explicitly overridden.
4. DELETE /sessions/{sessionId}/sets/{setId} removes only an unperformed set. Performed set deletion is 409. Do not renumber retained sets or change their IDs.
5. Keep a firstCompletion boolean in this workflow. 14 will add timer changes in this exact transaction; do not add an event bus or an empty timer service now.

Slice 11 - Performed-set verification

Gate

- Log 8 reps at 135.5 lb, correct values/type, and assert first completion time and targets are unchanged.
- Compare zero reps with untouched null reps; verify null weight versus explicit zero and both units.
- Reject negative/fractional reps, incomplete weights, unknown types, foreign IDs, and wrong session chains without writes.
- Append concurrently; positions are distinct. Extra targets are null and rest is resolved correctly.
- Delete an untouched set; retained IDs stay stable. Performed deletion is 409.
- Seed a final state; all writes return 409. Repeat logging and race against finalization again after 12 exists.

Acceptance

11.1 Recording performs a set. 11.2 Decimal/null weights work. 11.3 Zero differs from untouched. 11.4 Corrections preserve first time/targets. 11.5 Bounds and chain checks hold. 11.6 Final sessions are immutable. 11.7 Explicit units survive concurrent preference changes.

PROGRESS / SLICE 11

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 12 - Finish and browse history

Already exists

Performed sets, parent session locking, and state columns. Add 008_history_indexes.sql. Extend the sessions feature, not startup.

Add and implement

1. Complete/abandon actions lock the owned session. Completion requires at least one performed set; abandonment accepts an empty workout. Finalize status and finished_at once from TimeProvider.
2. Same final-state retry returns the original snapshot/time. Switching completed<->abandoned returns 409. All result/extra-set/delete mutations check this same state under the same lock.
3. Count performed sets by nonnull actual_reps/completed_at, including zero. Untouched sets stay null; partial completion does not fabricate results.
4. GET /sessions uses keyset paging by started_at DESC,id DESC. Limit defaults to 20, valid range 1-100. Validate status and a versioned base64 cursor containing the last UTC instant and int ID. Treat cursor data as untrusted and parameterize it; no cursor signing is needed for private filtered reads.
5. Apply user/status filters on every page and request limit+1 to determine nextCursor. Document paging over concurrent inserts; the cursor does not promise a frozen database-wide snapshot.

Keyset predicate - partial SQL fragment

```
WHERE user_id = @userId
      AND (started_at, id) < (@cursorStartedAt, @cursorId)
ORDER BY started_at DESC, id DESC
LIMIT @take;
```

For the first page omit the cursor predicate. Validate bounds and serialization precision so ties paginate correctly. A finalization after 14 also clears both timer fields; until then those fields do not exist.

Slice 12 - Verify lifecycle and paging

Gate

- Complete two of five sets; count two performed, preserve three null results. Zero-rep results count as performed.
- Empty completion returns 409; empty abandonment succeeds.
- Repeat each finalization and verify the original finishedAt. Switching final states and later mutations return 409.
- Race result writes, complete, and abandon; one serialized valid outcome persists. No write lands after finalization.
- Traverse several pages with tied startedAt values; no duplicates/omissions in a static fixture. Foreign history never appears.
- Invalid limit, status, or cursor returns 400 with no SQL interpolation. Verify nextCursor=null on the last page.
- Read 135.5 lb history after metadata/preferences change; snapshot targets, results, brands and units are intact.

Acceptance

12.1 Final status/time. 12.2 Partial completion. 12.3 Empty completion/abandon rules. 12.4 Same-state replay. 12.5 Stable private paging. 12.6 Immutable snapshots.

Carry forward

Previous performance in 13 reads these snapshots. Timer in 14 adds a small mutation to the existing first-result and finalization transactions; it does not create a parallel write pathway.

PROGRESS / SLICE 12

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 13 - Previous performance

Already exists

Completed sessions, snapshot DTOs, and the previous-completed index introduced with 008. No new table or stored previous-reps field.

Add and implement

1. GET /routines/{id}/previous-session first verifies the owned routine exists, then finds its latest completed session by finished_at DESC,id DESC. Active and abandoned sessions are excluded.
2. Reuse snapshot read assembly and return {previousSession:null} when no completed session exists. A foreign routine remains 404, not a null response.
3. Match repeated movements by sourceRoutineExerciseId. A new placement has no match even when it uses the same library exercise. Keep individual sets and original brands/weights.
4. Display conversion belongs at the presentation boundary: kilograms = pounds * 0.45359237m; pounds = kilograms / 0.45359237m. Convert from the stored original each time, then round display to two decimals AwayFromZero.
5. Preserve null/zero and fall back to original value/unit if decimal conversion overflows. The API continues returning original pairs; avoid an ambiguous converted value without its unit.

Slice 13 - Previous-performance verification

Gate

- Two completed workouts select the last finished, with ID tie-breaker. A newer abandoned/active session does not replace it.
- Repeated Bench Press placements with different results match their own source IDs. A newly created replacement placement gets no old match.
- No previous workout returns previousSession:null; foreign routine returns 404.
- Verify individual warmup/working results, zero reps, unperformed nulls, copied brands, and exact 135.5 lb.
- Presentation check: 100 lb shows 45.36 kg, then switching back still starts from stored 100 lb. Test null/zero and decimal overflow fallback.

Acceptance

13.1 Latest completed selection. 13.2 Active/abandoned excluded. 13.3 Placement identity matching. 13.4 Individual result fidelity. 13.5 Null history is normal. 13.6 Display never rewrites history.

PROGRESS / SLICE 13

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 14 - Persistent rest deadline

Already exists

First-completion detection in 11, parent session locks, TimeProvider, and finalization in 12. Add 009_rest_timer.sql and extend SessionDto with serverNow/restTimer.

Add and implement

1. On first result only, set `rest_ends_at=now+resolved seconds` and `rest_after_set_id=setId` in the same result transaction. Zero rest clears both. Another first-completed set replaces the timer; corrections do not restart it.
2. PUT `/sessions/{id}/rest-timer` stores the supplied UTC EndsAt and clears after-set attribution. Require a bound instant compatible with storage; a past deadline is valid and displays zero. Repeating an absolute deadline is idempotent.
3. DELETE clears both fields. Every timer change locks/checks the same owned in `_progress` session. Complete/abandon now clear the timer in their existing transactions.
4. Return `serverNow` from TimeProvider and the persisted deadline; return a `restTimer` only when `rest_ends_at` is present. Expiration needs no database write, background job, SignalR server, or server-side ticking loop.
5. Client computes `max(0,deadline-adjustedNow)`, using `serverNow` to estimate clock offset. Browser refresh/backgrounding must reconstruct from the deadline, not a decrementing counter saved each second.

Slice 14 - Timer verification

Gate

- Fixed-clock first result +120 seconds persists one deadline/afterSetId; zero clears it.
- Refresh shows the same deadline; correcting a result preserves it; another set replaces it.
- Manual absolute update/retry and cancel behave correctly. Manual update has no afterSetId.
- Expired deadlines remain readable and display zero. Test clock offset and background/resume on a real client when available.
- Finalization clears both fields; further timer writes return 409. Foreign session is 404.
- Direct SQL cannot reference a set from another session. Race result/finalization/timer writes and assert consistent final state.

Acceptance

14.1 Automatic timer. 14.2 Persistent deadline. 14.3 Correct correction/replacement. 14.4 Manual edit/cancel. 14.5 No expiry worker. 14.6 Finalization clears it.

PROGRESS / SLICE 14

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 15 - Reorder and archive templates

Already exists

Archive columns/indexes, owned parent locks, immutable snapshots, and full routine reads. No new schema is needed unless implementation reveals a concrete missing constraint.

Add and implement

1. Reorder takes exactly the active child IDs, once each. Lock routine parent and validate the complete owned list. Allocate temporary positive positions above the current maximum, then final 1..N positions in one transaction. Check int overflow before any update; reject safely rather than colliding.
2. DELETE planned set/placement archives the source row. Do not physically delete source rows referenced by historical sessions. New starts exclude archived children; old active/completed sessions retain copied values.
3. DELETE routine locks it, archives it, and clears schedule entries in one transaction. Same owned archive returns 204. Metadata changes/start/schedule writes reject the archived state.
4. DELETE exercise locks its owned row. Reject with 409 if an unarchived placement in an unarchived routine uses it. Creation of a placement already locks that same exercise; keep routine-before-exercise ordering where both locks are needed.
5. To replace a movement, archive old placement and create a new one. Library brand edits affect routine reads and future starts; they never update session snapshot rows.

Archive guard - partial SQL predicate

```
SELECT EXISTS (  
  SELECT 1 FROM routine_exercises re  
  JOIN routines r ON r.id = re.routine_id AND r.user_id = re.user_id  
  WHERE re.exercise_id = @exerciseId AND re.user_id = @userId  
    AND re.archived_at IS NULL AND r.archived_at IS NULL  
);
```

Execute after taking the exercise lock. A standalone NOT EXISTS followed by an update can race a new placement. Keep errors before writes when possible; rollback any rejected multi-command operation.

Slice 15 - Verify editing and historical fidelity

Gate

- Change name/description, 135 lb target to 140 lb, reps, types, rest, and brand. New starts see changes; previous active/completed sessions keep originals.
- Reorder placements and sets. Duplicate, missing, extra, foreign IDs, and overflow-risk temporary positions are rejected without changing order.
- Archive planned set/placement; future starts omit it, previous snapshots keep it. IDs are never reused.
- Archive a scheduled routine; assignments clear atomically. History stays readable; starts/metadata/schedule changes reject it.
- Exercise with active usage returns 409. After all active usage is removed, archive succeeds and historical references remain intact.
- Race exercise archive/placement attach, routine archive/schedule assignment, and reorder/start. Assert one valid serialized result and no mixed snapshot or duplicate position.

Acceptance

15.1 Template edits work. 15.2 Weight edits preserve snapshots. 15.3 Atomic reordering. 15.4 Removed source content stays historical. 15.5 Archive clears schedule. 15.6 Active snapshots also remain stable. 15.7 Concurrent writes/starts are consistent. 15.8 Brand history remains true. 15.9 Archive/attach race is closed.

Product milestone

The complete workout API is now implemented. A frontend can create templates, schedule routines, start/log/finish sessions, show prior results, and reconstruct timers. Infrastructure slices make this existing behavior operable on the Linux host.

PROGRESS / SLICE 15

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 16 - Prepare Linux production

Already exists

Local images, Compose network/volumes, configuration validation, and a complete tested API. Keep a single-host deployment; no Kubernetes or cloud service migration is implied.

Add and implement

1. Verify host CPU, supported Docker Engine/Compose, storage, and system service startup. Create a protected deployment directory with an explicit owner.
2. Supply production connection strings, Clerk issuer/claims, allowed origins, hostname, and image identities outside the repo/images. Restrict file permissions; do not emit resolved secrets through diagnostic commands.
3. Provision separate application and migrator DB roles. Application gets database connect/schema usage, table DML, and sequence privileges needed for identities; migrator owns schema/DDDL. Configure grants/default privileges under the actual role creating new tables. API must not have CREATE/DROP/ALTER rights.
4. Choose a supported Postgres major and approved release digest. Keep Postgres state on its named volume and record the matching restore-tool version.
5. Confirm runtime globalization/timezone support and trusted Clerk HTTPS from this host. Required invalid configuration fails startup; DB availability remains a readiness signal.

Gate

- Bring up the production configuration and verify architecture/versions and persistent data.
- Inspect permissions and tracked/build artifacts; verify secrets are absent from source/layers/logs.
- Application role can perform actual CRUD/identity inserts but cannot create/alter/drop a test table. Migrator role can apply a new script and grants continue to work.
- Invalid required configuration fails clearly without values. User timezone validation works in Linux.

Acceptance

16.1 Host is compatible. 16.2 Secrets are external/protected. 16.3 Configuration fails safely. 16.4 Roles have intended privileges. 16.5 Versions are controlled.

PROGRESS / SLICE 16

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 17 - HTTPS and browser access

Already exists

Internal API/db services and explicit frontend CORS/Clerk authorized-party validation. Add Caddy and trusted proxy settings for the actual host.

Add and implement

1. Point a trusted hostname to the host, configure Caddy's TLS/reverse proxy to api:8080, and publish only 80/443. Persist its certificate state.
2. Keep business paths /api/v1 unchanged. Health may be reached by an internal probe; do not expose diagnostic detail endpoints publicly.
3. Configure ASP.NET ForwardedHeaders before middleware that uses scheme/client IP. Trust only the actual proxy address/network and intended forwarded headers. Never clear trusted-proxy restrictions to accept any client [12].
4. Let the edge proxy own HTTPS redirect/certificate renewal. Internal API remains HTTP; do not introduce an internal redirect loop. Test the generated Location scheme through the proxy.
5. Test browser preflight and bearer requests with exact allowed origins. No wildcard credentialed CORS. Keep tokens/headers out of Caddy logs.

Gate

- An intended device trusts the hostname/certificate and can authenticate through HTTPS.
- HTTP redirects to HTTPS; direct API/DB ports are unreachable externally.
- Allowed-origin preflight succeeds; disallowed origin receives no permission. Authentication is still enforced regardless of Origin.
- Recreate proxy; certificate state remains and renewal is enabled.
- Spoofed forwarded headers from an untrusted path do not change trusted scheme/IP; generated Location works.

Acceptance

17.1 Trusted HTTPS. 17.2 HTTP redirect. 17.3 Private internal services. 17.4 Explicit browser origins. 17.5 Persistent certificates.

PROGRESS / SLICE 17

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 18 - Repeatable deployment

Already exists

Production host, image pair, migrator role, migration journal, readiness and HTTPS. Add one concrete runbook or small shell script for this host.

Add and implement

1. Lock deployments with native flock on one protected host lock file. Identify both images by one release ID and pinned digest. Pull/build before activation.
2. Take/verify the required backup and select a compatible migration plan. For breaking schema changes use a documented maintenance window; do not imply an automatic rollback can undo data loss.
3. Run a fresh one-shot migrator from the new release using docker compose run --rm migrations. Check exit code. A previous service_completed_successfully container is not evidence that this release ran migrations.
4. On success activate the API image, then gate on /health plus an authenticated representative API read/write through HTTPS. A connectivity probe alone cannot prove schema/API compatibility.
5. On failure leave the new API inactive. If the old API keeps serving while migrations run, all changes must be backward-compatible. Otherwise deliberately stop writes under maintenance. Retain the previous image/config and document compatible rollback or restore.

Slice 18 - Deployment verification

Gate

- Fresh install and seeded previous-release upgrade both follow the same script path.
- A new unapplied script runs exactly once; a failing script blocks new activation and does not pretend earlier committed scripts rolled back.
- Concurrent deploy attempts serialize. Verify interrupted deployment can be safely inspected and resumed.
- Upgrade preserves ownership, placements, snapshots, brands, preferences, exact decimals and original units.
- Rehearse compatible image rollback and incompatible-change restore. Never rewrite a journaled migration.

Acceptance

18.1 Fresh deployment. 18.2 New release runs its own migrator. 18.3 Failed migration blocks activation. 18.4 Data is preserved. 18.5 Deployment is serialized. 18.6 Recovery respects schema compatibility.

Legacy UUID/brand/pounds-only upgrade cases are conditional work only if actual retained legacy data exists; they are not part of the fresh-install critical path.

PROGRESS / SLICE 18

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 19 - Backup and restore

Already exists

Protected host configuration, controlled Postgres major, deployment runbook, and production data. Add a systemd timer/service or the host's existing scheduler; do not build an application backup worker.

Add and implement

1. Daily pg_dump custom-format archive with the matching supported tool version. Write to a temporary filename; check command success and pg_restore --list before marking the archive complete.
2. Record failures with a nonzero exit. Do not treat an empty/partial archive as valid. Retain seven days in an explicitly designated backup directory; restrict deletion to that directory.
3. Copy completed archives off-host to a retrievable protected destination. Protect deployment configuration/secrets separately; logical DB dumps do not replace host/role/config recovery.
4. Document restore into a separate empty database, including roles/grants, migration journal, application configuration, and the exact release image. Verify users/templates/snapshots, not just table counts.
5. Rehearse recovery without original containers and record measured duration. Identity sequence state is part of restore; an ordinary new insert must not collide.

Gate

- Backup a seeded DB; inspect archive and retrieve the off-host copy.
- Break DB authentication/destination; failure is visible and no incomplete file is retained as a successful backup.
- Test seven-day boundary retention inside a disposable directory.
- Restore separately and verify accounts, schedule, source IDs, snapshots, zero/null reps/weights, 135.5 lb, 60 kg, original units, null/named brands and preferences.
- Create a new row after restore to check sequence state. Recover protected config without relying on original containers; record recovery time.

Acceptance

19.1 Usable daily archives. 19.2 Visible failure. 19.3 Retention and off-host retrieval. 19.4 Data fidelity. 19.5 Independent recovery. 19.6 Config/roles recoverable. 19.7 Brands preserved. 19.8 Identities/units preserved.

PROGRESS / SLICE 19

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Slice 20 - Recovery and diagnostics

Already exists

Working release/deploy/backup/restore procedures. Add bounded logging configuration and a short operator runbook. Reuse ILogger and native Docker log rotation first.

Add and implement

1. Enable Docker at host startup and restart policies for long-running db/api/proxy. Migrator remains one-shot. Reboot/crash checks use existing data volumes.
2. Bound connection, command, shutdown and outbound metadata operations. Verify readiness moves 503->200 after DB recovery; do not restart every API process for one failed connection.
3. Log request trace ID, method/route, status, duration, and safe error classification. Never log JWTs, authorization headers, connection strings, or full bodies/SQL parameters. Use structured ILogger fields, not string-concatenated secret data.
4. Keep unexpected faults as sanitized 500. Translate positively identified dependency failures to 503; expected uniqueness/state conflicts stay 409. If IExceptionHandler is introduced, handle .NET 10's diagnostic suppression deliberately, with safe logging exactly once [13].
5. Set Docker log max-size/max-file limits. Document health, logs, restart, deploy, backup, restore, and last successful recovery evidence in one runbook.

Slice 20 - Recovery verification

Gate

- Reboot the host and crash API/proxy; services recover, volumes remain intact, migrations do not loop.
- Restart Postgres; health becomes 503 and recovers. Interrupt start/result/timer transactions and verify full commit or rollback; safe retry preserves idempotency and units.
- Repeat SIGTERM during real writes and confirm graceful shutdown or rollback.
- Trigger rotation; disk use stays bounded. Inspect logs for useful trace/status/duration and absence of test secrets.
- Follow the runbook from a clean operator session; restore the off-host backup independently.

Acceptance

20.1 Reboot recovery. 20.2 Crash restart. 20.3 DB reconnection. 20.4 Atomic interrupted writes. 20.5 Useful safe logs. 20.6 Bounded disk usage. 20.7 Routine operations preserve data.

PROGRESS / SLICE 20

☐ Implementation complete

☐ Verification passed

Status: Planned / Building / Blocked / Done

Review date (YYYY-MM-DD)

Planned

Evidence / failures / follow-ups

Final release gate

Run against the Linux host through HTTPS. Retain evidence before marking the app release-ready.

- Sign in with two accounts; verify provisioning, claim restrictions, private lists and 404 ownership boundaries.
- Create all five routines and weekday assignments with descriptions, optional brands, repeated movements, warmup/working sets, rest overrides, and mixed lb/kg targets.
- Start with a retry key, retry concurrently, log zero and nonzero reps at 135.5 lb/60 kg, correct a result, add/remove an extra set, reconstruct the timer, and finish partially.
- Edit template targets and library brand; old active/completed snapshots remain identical and new snapshots use current metadata. Previous performance matches source placements.
- Switch imperial/metric preferences; display converts from original stored weights. Verify explicit units during preference changes and preserve null versus zero.
- Recreate containers/reboot; validate persistent state, readiness recovery, trusted HTTPS, private internal ports, bounded safe logs and interrupted-write rollback.
- Restore an off-host backup separately, then insert a new row. Verify identities, source relationships, units, preferences, brands, timers, grants and migration journal.
- Rehearse failed new migration, serialized deployments, compatible rollback and incompatible-change recovery. Record the tested release digests.

Verification record for this revised guide

This document separates planned acceptance checks from checks performed while authoring it. Regenerating the PDF does not execute the app, SQL migrations, Clerk integration, Docker deployment, or restore. Any recipe compilation or PDF/form checks actually performed are reported with the delivered artifact; all product gates remain work for their slices.

- Complete slice-one API and migrator recipes compiled in isolated scratch projects with .NET SDK 10.0.100 and the listed package pins: zero warnings and errors. The current workspace application was not changed or certified.
- HTTP smoke checks passed: test-only v1 routing, unversioned/v2 404 ProblemDetails, bounded database-unavailable readiness 503, development OpenAPI, secret-free test logs, and missing-configuration failure for both executables.
- PDF checks passed: 62 pages and bookmarks, 100 canonical progress fields and matching widgets, form appearances, save/reopen persistence, content bounds, and rendered-page inspection.
- Real-Postgres migration/rollback checks, live Clerk integration, containers, host deployment and recovery were not executed during this document revision.

Begin with 01

Correct SDK selection, compose startup through two service extensions, add native readiness, establish the users baseline/migrator, and pass the focused integration gate. Then proceed to images and Compose. No future feature registrations are needed to get the foundation running.

API reference - route catalog

All business paths below are prefixed /api/v1 and protected after 04. IDs are positive ints. The path variable names distinguish library exercise, routine placement, and session exercise. This revision intentionally uses 400 for invalid bound input rather than the old 422 contract.

Slice	Method / path	Request -> result
01	GET /health (unprefixed)	Public readiness 200/503 text
04	GET /me	UserDto
04	PUT /me/preferences	PreferencesRequest -> UserDto
05	GET /exercises	includeArchived=false -> ExerciseDto[]
05	POST /exercises	ExerciseWriteRequest -> 201 ExerciseDto
05	GET /exercises/{id}	ExerciseDto
05	PUT /exercises/{id}	ExerciseWriteRequest -> ExerciseDto
15	DELETE /exercises/{id}	Archive 204; active usage 409
06	GET /routines	includeArchived=false -> RoutineSummaryDto[]
06	POST /routines	NameDescriptionRequest -> 201 RoutineDto
06	GET /routines/{id}	Ordered RoutineDto
06	PUT /routines/{id}	NameDescriptionRequest -> RoutineDto
15	DELETE /routines/{id}	Archive/clear schedule -> 204
07	POST /routines/{id}/exercises	PlacementCreateRequest -> 201 placement
07	PUT /routines/{id}/exercises/{placementId}	PlacementUpdateRequest -> placement
15	DELETE /routines/{id}/exercises/{placementId}	Archive -> 204
15	PUT /routines/{id}/exercises/order	ReorderRequest -> RoutineDto

Use versioned CreatedAtAction links for creates. PUT replaces the editable fields represented by its request.

Route mismatch/foreign resource is 404; owned archived state is 409 for writes. Repeated owned archives return 204.

API reference - sets, schedule, sessions

Slice	Method / path (prefix /api/v1)	Request -> result
08	POST /routines/{id}/exercises/{placementId}/sets	RoutineSetRequest -> 201 set
08	PUT /routines/{id}/exercises/{placementId}/sets/{setId}	RoutineSetRequest -> set
15	DELETE /routines/{id}/exercises/{placementId}/sets/{setId}	Archive -> 204
15	PUT /routines/{id}/exercises/{placementId}/sets/order	ReorderRequest -> placement
09	GET /schedule	ScheduleEntryDto[]
09	PUT /schedule/{weekday}	ScheduleRequest -> ScheduleEntryDto
09	DELETE /schedule/{weekday}	Idempotent clear -> 204
10	POST /routines/{id}/sessions	StartSessionRequest + key -> 201/200 SessionDto
10	GET /sessions/{id}	SessionDto
12	GET /sessions	limit/cursor/status -> PageResponse
11	PUT /sessions/{sessionId}/sets/{setId}/result	SetResultRequest -> SessionSetDto
11	POST /sessions/{sessionId}/exercises/{sessionExerciseId}/sets	ExtraSetRequest -> 201 set
11	DELETE /sessions/{sessionId}/sets/{setId}	Unperformed only -> 204
12	POST /sessions/{id}/complete	SessionDto
12	POST /sessions/{id}/abandon	SessionDto
13	GET /routines/{id}/previous-session	PreviousSessionResponse
14	PUT /sessions/{id}/rest-timer	RestTimerRequest -> SessionDto
14	DELETE /sessions/{id}/rest-timer	Clear -> 204

35 product routes total: 34 business routes and one public readiness route. Development OpenAPI is tooling, not an additional product operation. Unsupported URL versions do not execute v1 actions.

Timestamps are UTC ISO 8601, scheduledDate is YYYY-MM-DD. Warmup/working types, in_progress/completed/abandoned states, imperial/metric preferences, and lb/kg units are explicit lowercase strings. Null actualReps is untouched; zero is performed. Null weight is unrecorded; zero weight is recorded zero.

Contract reference - requests

These are wire field definitions, not duplicate types to paste into every feature. Use DataAnnotations on actual request types. Required numeric inputs must detect omission (nullable + [Required], then range validation); do not accidentally turn missing reps/value into valid zero. Optional fields have explicit null semantics.

```
PreferencesRequest
  timeZone: required IANA string (UTC allowed)
  measurementSystem: required imperial|metric
NameDescriptionRequest
  name: required nonblank string
  description: optional string -> ""
ExerciseWriteRequest
  name, description as above
  brandName: optional string -> trimmed null or <=100 characters
PlacementCreateRequest
  exerciseId: required positive int (library ID)
  position: required positive int
  description: optional string -> ""
  defaultRestSeconds: required int >=0
PlacementUpdateRequest
  position, description, defaultRestSeconds (exerciseId is immutable)
WeightInput
  value: required decimal >=0; unit: required lb|kg
RoutineSetRequest
  position: required int >0; setType: required warmup|working
  targetRepsMin: required int >0; targetRepsMax: required int >=min
  targetWeight: optional complete WeightInput; restSeconds: optional int >=0
ReorderRequest
  orderedIds: required int[]; exact active children once each
ScheduleRequest
  routineId: required positive int
StartSessionRequest
  scheduledDate: optional date
  header Idempotency-Key: required 1-128 visible ASCII characters
SetResultRequest
  actualReps: required int >=0; setType: required warmup|working
  actualWeight: optional complete WeightInput
ExtraSetRequest
  setType: required warmup|working; restSeconds: optional int >=0
RestTimerRequest
  endsAt: required DateTimeOffset compatible with UTC storage
```

Omitted/null actualWeight clears its recorded pair on replacement. An absent targetWeight means no planned weight. A valid zero needs value=0 and a unit. Normalize optional description/brand once before writing; reuse those normalized values for validation and SQL.

Contract reference - responses

Nested collections are ordered and nonnull. UTC instant fields map from internal UTC DateTime (or zero-offset DateTimeOffset at the boundary). Database rows stay separate from these nested objects.

```
UserDto: id, clerkUserId, timeZone, measurementSystem
WeightDto: value(decimal), unit(lb|kg)
ExerciseDto: id, name, description, brandName?, archivedAt?
RoutineSummaryDto: id, name, description, archivedAt?
RoutineSetDto:
  id, position, setType, targetRepsMin, targetRepsMax,
  targetWeight?, restSeconds?
RoutineExerciseDto:
  id(placement), exerciseId(library), exerciseName, brandName?,
  position, description, defaultRestSeconds, sets[]
RoutineDto:
  id, name, description, archivedAt?, exercises[]
ScheduleEntryDto: weekday, routineId, routineName
SessionSetDto:
  id, position, setType, targetRepsMin?, targetRepsMax?, targetWeight?,
  actualReps?, actualWeight?, restSeconds, completedAt?
SessionExerciseDto:
  id(session child), exerciseId(library), sourceRoutineExerciseId,
  exerciseName, brandName?, position, description,
  defaultRestSeconds, sets[]
SessionSummaryDto:
  id, routineId, routineName, status, scheduledDate?,
  startedAt, finishedAt?, performedSetCount
RestTimerDto: endsAt, afterSetId?
SessionDto:
  id, routineId, routineName, routineDescription, status,
  scheduledDate?, startedAt, finishedAt?, serverNow, restTimer?, exercises[]
PreviousSessionResponse: previousSession?
PageResponse<SessionSummaryDto>: items[], nextCursor?
```

Rest fields/serverNow are added to session responses in 14. If API consumers already exist by then, adding these optional fields is additive; do not rename or reinterpret existing fields. Maintain OpenAPI response metadata for actual action outcomes as the feature is implemented.

RoutineExerciseDto brand/name reflect the current library. SessionExerciseDto fields reflect only the copied snapshot. Performed count includes zero reps. History uses snapshot routineName, not a join that silently changes old workouts.

Primary references

Documentation checked while rebuilding this guide. Links support the framework mechanisms; the workout rules and slice order are this project's design decisions.

1. Microsoft: controller API behavior, model validation and ProblemDetails.
<https://learn.microsoft.com/en-us/aspnet/core/web-api/?view=aspnetcore-10.0>
2. .NET Foundation: current ASP.NET API Versioning project and package lineage.
<https://github.com/dotnet/aspnet-api-versioning>
3. API Versioning: URL segment route matching.
<https://dotnet.github.io/aspnet-api-versioning/aspnet-core/how-to/version-by-url.html>
4. Npgsql: data sources, pooling, disposal and transactions. <https://www.npgsql.org/doc/basic-usage.html>
5. Dapper: query mapping, parameters and CommandDefinition. <https://github.com/DapperLib/Dapper>
6. Microsoft: SDK selection with global.json. <https://learn.microsoft.com/en-us/dotnet/core/tools/global-json>
7. Microsoft: native ASP.NET Core health checks.
<https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/health-checks?view=aspnetcore-10.0>
8. DbUp: transaction strategies. <https://dbup.readthedocs.io/en/latest/more-info/transactions/>
9. Docker: official Postgres image configuration and version 18 volume layout. https://hub.docker.com/_/postgres
10. Clerk: session-token verification and authorized parties.
<https://clerk.com/docs/guides/sessions/manual-jwt-verification>
11. PostgreSQL: transaction isolation. <https://www.postgresql.org/docs/current/transaction-iso.html>
12. Microsoft: trusted proxies and forwarded headers.
<https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/proxy-load-balancer?view=aspnetcore-10.0>
13. Microsoft: .NET 10 handled-exception diagnostic suppression. <https://learn.microsoft.com/en-us/aspnet/core/breaking-changes/10/exception-handler-diagnostics-suppressed?view=aspnetcore-10.0>
14. NuGet: checked foundation versioning and OpenAPI packages.
<https://www.nuget.org/packages/Asp.Versioning.Mvc/10.2.1> and
<https://www.nuget.org/packages/Microsoft.AspNetCore.OpenApi/10.0.12>

The following SQL reference is split by first-use migration. Fresh installs execute that same sequence; it is not a separate baseline that should be run alongside the individual scripts. DB execution remains an implementation verification gate.

Migration reference - 001_users.sql / introduced in 01

```
CREATE TABLE users (  
    id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    clerk_user_id     text NOT NULL UNIQUE,  
    time_zone         text NOT NULL DEFAULT 'UTC',  
    measurement_system text NOT NULL DEFAULT 'imperial'  
                        CHECK (measurement_system IN ('imperial', 'metric')),  
    created_at        timestampz NOT NULL DEFAULT now(),  
    CHECK (btrim(clerk_user_id) <> '')  
);
```

Migration reference - 002_exercises.sql / introduced in 05

```
CREATE TABLE exercises (  
    id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id           integer NOT NULL REFERENCES users(id),  
    name              text NOT NULL,  
    description        text NOT NULL DEFAULT '',  
    brand_name         text CHECK (brand_name IS NULL OR  
                                (btrim(brand_name) <> '' AND length(brand_name) <= 100)),  
    archived_at        timestamptz,  
    created_at         timestamptz NOT NULL DEFAULT now(),  
    UNIQUE (id, user_id),  
    CHECK (btrim(name) <> '')  
);  
CREATE INDEX exercises_active_list  
    ON exercises(user_id, name)  
    WHERE archived_at IS NULL;
```

Migration reference - 003_routines.sql / introduced in 06

```
CREATE TABLE routines (  
    id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id           integer NOT NULL REFERENCES users(id),  
    name              text NOT NULL,  
    description       text NOT NULL DEFAULT '',  
    archived_at       timestampz,  
    created_at        timestampz NOT NULL DEFAULT now(),  
    UNIQUE (id, user_id),  
    CHECK (btrim(name) <> '')  
);  
CREATE INDEX routines_active_list  
    ON routines(user_id, name)  
    WHERE archived_at IS NULL;
```

Migration reference -

004_routine_exercises.sql / introduced in 07

```
CREATE TABLE routine_exercises (  
    id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id           integer NOT NULL REFERENCES users(id),  
    routine_id        integer NOT NULL,  
    exercise_id       integer NOT NULL,  
    position          integer NOT NULL CHECK (position > 0),  
    description       text NOT NULL DEFAULT '',  
    default_rest_seconds integer NOT NULL DEFAULT 120  
                        CHECK (default_rest_seconds >= 0),  
    archived_at       timestampz,  
    UNIQUE (id, user_id),  
    FOREIGN KEY (routine_id, user_id)  
        REFERENCES routines(id, user_id),  
    FOREIGN KEY (exercise_id, user_id)  
        REFERENCES exercises(id, user_id)  
);  
CREATE UNIQUE INDEX routine_exercises_active_position  
    ON routine_exercises(routine_id, position)  
    WHERE archived_at IS NULL;
```

Migration reference - 005_routine_sets.sql / introduced in 08

```
CREATE TABLE routine_sets (  
    id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id           integer NOT NULL REFERENCES users(id),  
    routine_exercise_id integer NOT NULL,  
    position          integer NOT NULL CHECK (position > 0),  
    set_type          text NOT NULL  
                        CHECK (set_type IN ('warmup', 'working')),  
    target_reps_min   integer NOT NULL CHECK (target_reps_min > 0),  
    target_reps_max   integer NOT NULL,  
    target_weight     numeric CHECK (target_weight >= 0  
                        AND target_weight < 'Infinity'::numeric),  
    target_weight_unit text CHECK (target_weight_unit IN ('lb', 'kg')),  
    CHECK ((target_weight IS NULL) = (target_weight_unit IS NULL)),  
    rest_seconds      integer CHECK (rest_seconds >= 0),  
    archived_at       timestamptz,  
    CHECK (target_reps_max >= target_reps_min),  
    FOREIGN KEY (routine_exercise_id, user_id)  
        REFERENCES routine_exercises(id, user_id)  
);  
CREATE UNIQUE INDEX routine_sets_active_position  
    ON routine_sets(routine_exercise_id, position)  
    WHERE archived_at IS NULL;
```

Migration reference -

006_routine_schedule.sql / introduced in 09

```
CREATE TABLE routine_schedule (  
    user_id          integer NOT NULL REFERENCES users(id),  
    weekday          smallint NOT NULL CHECK (weekday BETWEEN 1 AND 7),  
    routine_id       integer NOT NULL,  
    PRIMARY KEY (user_id, weekday),  
    FOREIGN KEY (routine_id, user_id)  
        REFERENCES routines(id, user_id)  
);  
CREATE INDEX routine_schedule_routine  
    ON routine_schedule(routine_id);
```


Migration reference - 007_sessions.sql / part 1 / introduced in 10

Combine parts 1-4 in this order in one embedded 007_sessions.sql. These page breaks are for reading, not separate migration files.

```
CREATE TABLE workout_sessions (  
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  user_id integer NOT NULL REFERENCES users(id),  
  routine_id integer NOT NULL,  
  -- Copied template metadata.  
  routine_name text NOT NULL,  
  routine_description text NOT NULL DEFAULT '',  
  scheduled_date date,  
  status text NOT NULL DEFAULT 'in_progress'  
  CHECK (  
    status IN (  
      'in_progress',  
      'completed',  
      'abandoned'  
    )  
  ),  
  started_at timestamptz NOT NULL DEFAULT now(),  
  finished_at timestamptz,  
  start_request_key text NOT NULL  
  CHECK (length(start_request_key) BETWEEN 1 AND 128),  
  UNIQUE (id, user_id),  
  UNIQUE (user_id, start_request_key),  
  FOREIGN KEY (routine_id, user_id)  
    REFERENCES routines(id, user_id),  
  CHECK (  
    (status = 'in_progress' AND finished_at IS NULL)  
    OR  
    (status IN ('completed', 'abandoned')  
     AND finished_at IS NOT NULL)  
  ),  
  CHECK (  
    finished_at IS NULL OR finished_at >= started_at  
  )  
);
```

Migration reference - 007_sessions.sql / part 2 / same script

Combine parts 1-4 in this order in one embedded 007_sessions.sql. These page breaks are for reading, not separate migration files.

```
CREATE TABLE session_exercises (  
    id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id integer NOT NULL REFERENCES users(id),  
    session_id integer NOT NULL,  
    exercise_id integer NOT NULL,  
    source_routine_exercise_id integer NOT NULL,  
    -- Copied exercise metadata.  
    exercise_name text NOT NULL,  
    brand_name text CHECK (brand_name IS NULL OR  
        (btrim(brand_name) <> ''  
        AND length(brand_name) <= 100)),  
    description text NOT NULL DEFAULT '',  
    position integer NOT NULL CHECK (position > 0),  
    default_rest_seconds integer NOT NULL  
        CHECK (default_rest_seconds >= 0),  
    UNIQUE (session_id, position),  
    UNIQUE (id, session_id, user_id),  
    FOREIGN KEY (session_id, user_id)  
        REFERENCES workout_sessions(id, user_id),  
    FOREIGN KEY (exercise_id, user_id)  
        REFERENCES exercises(id, user_id),  
    FOREIGN KEY (source_routine_exercise_id, user_id)  
        REFERENCES routine_exercises(id, user_id)  
);
```

Migration reference - 007_sessions.sql / part 3 / same script

Combine parts 1-4 in this order in one embedded 007_sessions.sql. These page breaks are for reading, not separate migration files.

```
CREATE TABLE session_sets (
  id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id           integer NOT NULL REFERENCES users(id),
  session_id        integer NOT NULL,
  session_exercise_id integer NOT NULL,
  position          integer NOT NULL CHECK (position > 0),
  set_type          text NOT NULL
                    CHECK (set_type IN ('warmup', 'working')),
  -- Copied targets; extra sets may have no rep target.
  target_reps_min   integer,
  target_reps_max   integer,
  target_weight     numeric CHECK (target_weight >= 0
                                AND target_weight < 'Infinity'::numeric),
  target_weight_unit text CHECK (target_weight_unit IN ('lb', 'kg')),
  CHECK ((target_weight IS NULL) = (target_weight_unit IS NULL)),
  -- Actual performance.
  actual_reps       integer CHECK (actual_reps >= 0),
  actual_weight     numeric CHECK (actual_weight >= 0
                                AND actual_weight < 'Infinity'::numeric),
  actual_weight_unit text CHECK (actual_weight_unit IN ('lb', 'kg')),
  CHECK ((actual_weight IS NULL) = (actual_weight_unit IS NULL)),
  completed_at      timestampz,
  -- Resolved when the set is created.
  rest_seconds      integer NOT NULL CHECK (rest_seconds >= 0),
  UNIQUE (session_exercise_id, position),
  UNIQUE (id, session_id, user_id),
  FOREIGN KEY (session_exercise_id, session_id, user_id)
    REFERENCES session_exercises(id, session_id, user_id),
  CHECK (
    (target_reps_min IS NULL AND target_reps_max IS NULL)
    OR
    (
      target_reps_min IS NOT NULL
      AND target_reps_max IS NOT NULL
      AND target_reps_min > 0
      AND target_reps_max >= target_reps_min
    )
  ),
  CHECK (
    (actual_reps IS NULL AND completed_at IS NULL)
    OR
    (actual_reps IS NOT NULL AND completed_at IS NOT NULL)
  )
);
```

Migration reference - 007_sessions.sql / part 4 / same script

Combine parts 1-4 in this order in one embedded 007_sessions.sql. These page breaks are for reading, not separate migration files.

```
ALTER TABLE session_sets ADD CONSTRAINT session_sets_weight_requires_result
    CHECK (actual_reps IS NOT NULL OR actual_weight IS NULL);
CREATE INDEX session_sets_session
    ON session_sets(session_id);
```

Session targets/results and lifecycle exist in 10. Their endpoints arrive in 11-12. The extra check prevents a stored weight on an untouched set. No timer columns exist until 009.

Migration reference - 008_history_indexes.sql / introduced in 12

```
CREATE INDEX workout_sessions_history
  ON workout_sessions(user_id, started_at DESC, id DESC);
CREATE INDEX workout_sessions_previous_completed
  ON workout_sessions(
    user_id,
    routine_id,
    finished_at DESC,
    id DESC
  )
WHERE status = 'completed';
```

Migration reference - 009_rest_timer.sql / introduced in 14

```
ALTER TABLE workout_sessions
  ADD COLUMN rest_ends_at timestamptz,
  ADD COLUMN rest_after_set_id integer,
  ADD CONSTRAINT workout_sessions_timer_pair CHECK (
    rest_ends_at IS NOT NULL OR rest_after_set_id IS NULL),
  ADD CONSTRAINT workout_sessions_final_timer CHECK (
    status = 'in_progress'
    OR (rest_ends_at IS NULL AND rest_after_set_id IS NULL)),
  ADD CONSTRAINT workout_sessions_rest_set_fk
    FOREIGN KEY (rest_after_set_id, id, user_id)
    REFERENCES session_sets(id, session_id, user_id);
```

All these statements execute through the same ordered migration journal. Timer attribution must reference a set from the exact session and owner. Finalization must clear both fields.